

Curso Inicial Front-End

Clase 19: JS Avanzado:

Gestión de Estados y

Práctica

Módulo 2: Dominio de JavaScript y Lógica de Programación



Presentación

Internet no es instantáneo. En esta clase aprenderemos a gestionar el tiempo mediante la **Asincronía**. Descubriremos cómo pedir datos a servidores externos sin congelar la pantalla del usuario. Dominaremos las **Promesas** y aprenderemos a usar `fetch` para consumir APIs reales, entendiendo los verbos HTTP y el manejo profesional de errores con `try-catch`.



Objetivos

- Comprender la naturaleza asíncrona de **JavaScript** y el uso de Promesas.
- Dominar el consumo de APIs mediante `fetch` y la sintaxis moderna `async/await`.
- Identificar y utilizar correctamente los verbos HTTP (GET, POST, PUT, DELETE).



Bloques Temáticos

- Tema 1: Lógica de actualización de estados en el DOM antes de la transición a React.
 - El Problema del Tiempo: El Mozo del Restaurante
 - Las Promesas (Promises): El Beeper de **JS**
 - Fetch: Lanzando el anzuelo a Internet
 - Gestión del Estado en el DOM (El Puente hacia **React**)

Hasta ahora, todos nuestros proyectos han vivido en una burbuja. Guardamos los datos de los usuarios en variables o en el `localStorage` de nuestra propia máquina. Pero una red social real no guarda sus fotos en tu navegador; las pide prestadas a una supercomputadora central de Facebook (un **Servidor**).

Hoy romperemos la burbuja. Aprenderemos cómo hacer que nuestro código le hable a servidores externos a través de internet para pedirles información fresca usando **APIs** (Interfaces de comunicación), y cómo lidiar con el problema de la lentitud de la red.

 **Recurso Oficial:** [MDN Web Docs - Trabajando con Promesas y Fetch](#)

Tema 1: Lógica de actualización de estados en el DOM antes de la transición a React.

1. El Problema del Tiempo: El Mozo del Restaurante

Internet no es instantáneo. Descargar una lista de 500 películas desde un servidor en Estados Unidos puede tomar 2 segundos.

Imagina que el motor de **JavaScript** es un mozo en un restaurante.

- **Código Sincrónico (Bloqueante):** El mozo lleva tu pedido a la cocina y se queda congelado frente al chef durante 30 minutos esperando a que la comida esté lista. Durante esos 30 minutos, las demás mesas del restaurante son ignoradas. Si **JS** hiciera esto, toda tu página web se "congelaría" y el usuario no podría hacer clic en nada mientras baja la película.
- **Código Asíncrono (Inteligente):** El mozo lleva tu pedido a la cocina, le dejan un "Beeper / Timbre" en la mano, y él se va inmediatamente a atender a otras mesas. Cuando la comida está lista, el timbre vibra, y el mozo sabe que debe ir a buscar el plato. ¡El restaurante jamás se detuvo!

En lenguaje simple: **JavaScript** solo puede hacer una cosa a la vez. Para no trabarse, le pide ayuda al navegador para tareas lentas (como bajar una foto). El navegador hace el trabajo sucio y, cuando termina, avisa al Event Loop para que **JS** retome esa tarea.

Traducción Técnica: **JavaScript** es un lenguaje **Single-Threaded** (de un solo hilo), lo que significa que solo puede ejecutar una instrucción a la vez. Sin embargo, delega las operaciones costosas (como el acceso a red o disco) a las **Web APIs** del navegador. Cuando una operación asíncrona finaliza, su callback se encola en la **Callback Queue** o **Microtask Queue** (en el caso de las Promesas). El **Event Loop** monitorea el Call Stack; cuando este se vacía, empuja la siguiente tarea de la cola al stack para su ejecución, permitiendo la concurrencia sin bloquear el hilo principal.

A este mecanismo técnico que le permite a **JS** seguir trabajando mientras la descarga ocurre en segundo plano se le llama **Event Loop (Bucle de Eventos)**.

¿Qué es una Promesa? (Promise)

En **JavaScript**, una **Promesa** es un objeto que representa una operación que aún no ha terminado pero que terminará en el futuro (con éxito o error). Es como cuando pides comida: te dan un ticket (la promesa). El ticket no es la pizza, pero es la garantía de que la pizza llegará.

El pasado vs El presente: `.then()` vs `async/await`

Antes usábamos el encadenamiento de métodos. Es importante que lo conozcas porque verás mucho código antiguo así: - `.then()`: Lo que se ejecuta si la promesa tiene éxito. - `.catch()`: Lo que se ejecuta si la promesa falla.

Hoy usamos `try...catch`: Es el bloque de seguridad moderno. Si algo falla dentro del `try`, el código salta inmediatamente al `catch` sin romper la aplicación.

2. El Protocolo HTTP: Los Verbos del Servidor

Cuando usamos `fetch()`, nos estamos comunicando mediante verbos que definen nuestra intención: - **GET**: "Dame información" (ej: ver lista de

productos). - **POST:** "Guarda esto nuevo" (ej: crear un usuario). - **PUT / PATCH:** "Modifica esto que ya existe". - **DELETE:** "Borra esto".

Librerías Externas: El caso de Axios

Aunque usaremos `fetch()` (nativo del navegador), en la industria verás mucho **Axios**. Es una librería que facilita las peticiones, pero requiere instalación. Nosotros dominaremos `fetch` primero para entender las bases.

3. Fetch: Lanzando el anzuelo a Internet

`fetch()` ("ir a buscar", en inglés) es la función nativa que nos regala el navegador para hacer peticiones HTTP hacia servidores externos y APIs.

La forma moderna: `async` / `await`

Para trabajar con Promesas, usamos dos palabras mágicas que hacen que el código se lea de forma hermosa y estructurada.

Mapeo de Sintaxis (Desglosando la magia):

```
/* 1. La palabra 'async' le avisa al sistema: "Oye, adentro de esta
función va a haber pausas de tiempo, prepárate". */
async function pedirPelículas() {

    // El bloque 'try/catch' es un escudo anti-caídas. Si algo falla en
    'try', JS salta automáticamente a 'catch' sin colgar la página.
    try {
```

```
/* 2. 'await' es el freno de mano temporal. Le dice al mozo:
"Lanza el Fetch y PON EN PAUSA esta línea hasta que lleguen los datos.
Pero tú ve a atender el resto del navegador". */
const respuestaCruda = await
fetch("https://api.cine.com/peliculas");

/* 3. ¡PELIGRO VITAL! Fetch es tonto. Si el servidor responde
"Error 404: Película No Encontrada", Fetch lo considera un éxito de
comunicación. Tenemos que frenarlo a mano usando '.ok'. */
if (!respuestaCruda.ok) {
    throw new Error("El servidor respondió con un error de
status.");
}

/* 4. Descriptación: La respuestaCruda viene en un formato de
texto plano pesado. Tenemos que usar '.json()' para transformarlo en un
[Array] real de JS. Como descriptar toma tiempo matemático, también
requiere un 'await'. */
const peliculasFinales = await respuestaCruda.json();

console.log("¡Carga terminada!", peliculasFinales);

} catch (error) {
    // Si se cortó tu internet, el error cae directo aquí.
    console.error("Algo salió terriblemente mal:", error.message);
}
}
```

🛑 **Checkpoint de Comprensión:** Estás programando una página del clima usando la herramienta `fetch()`. De repente, por un error del administrador de la base de datos, la API del clima es borrada. Cuando tu `fetch()` vaya a golpear esa puerta, el servidor le responderá un famoso código HTTP de error: el **404 Not Found**. Si no pusiste el `if (!respuestaCruda.ok) throw new Error(...)` en tu código, ¿crees que la ejecución caerá en el bloque verde de éxito y romperá tu programa más abajo, o caerá en el bloque rojo `catch` automáticamente para protegerte? (Pista: Lee la alerta de **PELIGRO VITAL** del código de arriba).

4. Gestión del Estado en el DOM (El Puente hacia React)

Conectar a internet y descargar datos es solo el 50% del trabajo. El otro 50% es **dibujarlos** en la pantalla.

En aplicaciones complejas, ya no disparamos `innerHTML` a lo loco. Usamos una **Variable de Estado**. Es una variable global que actúa como la "Única Fuente de Verdad". Cada vez que esa variable cambia, disparamos una función que "Dibuja" el DOM.

Ejemplo de Arquitectura Profesional:

```
// 1. EL ESTADO: La fuente de la verdad
let usuariosSistema = [];

// 2. EL PINTOR: Su único trabajo es mirar al estado y redibujar el HTML
function renderizarUI() {
  const contenedor = document.getElementById("lista");
  contenedor.innerHTML = ""; // Limpiamos el lienzo viejo

  // Por cada usuario en el Estado, creamos un elemento en pantalla
  usuariosSistema.forEach(user => {
    const li = document.createElement("li");
    li.textContent = user.nombre;
    contenedor.appendChild(li);
  });
}

// 3. EL DESCARGADOR: Modifica el estado
async function iniciarApp() {
  // Pedimos datos...
  const res = await fetch("https://api.usuarios.com");
  const data = await res.json();

  // Inyectamos los datos frescos en nuestra variable de Estado central
  usuariosSistema = data;

  // Le gritamos al pintor que los datos cambiaron, para que redibuje
  renderizarUI();
}
```

*Si entiendes este código de arriba, estás 100% listo para empezar **React.js** en la próxima clase.*

Documentación Oficial (Referencias)

- **HTML:** <https://developer.mozilla.org/es/docs/Web/HTML>
- **JS:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **JavaScript:** <https://developer.mozilla.org/es/docs/Web/JavaScript>
- **React:** <https://es.react.dev/>